

Midterm Project Report

1. Introduction and Objectives

Modern malware increasingly evades static, signature based detection through packing, obfuscation, and polymorphism. Behavioural (dynamic) analysis is more resilient to these evasions. Two behavioural signals are especially discriminative which are the sequence of Windows API calls a program issues and the network traffic it generates.

The objective of this project is to build a hybrid malware classifier that fuses both signals. Concretely, the work is divided into three steps:

1. Capture the API call sequence of each sample from a controlled execution
2. Capture the network traffic (PCAP) from the same execution
3. Combine the two feature types into a single hybrid classifier.

All malware is executed exclusively inside an isolated virtual machine on a network that has no route to the public internet.

2. Sandbox Architecture

The analysis environment was built on the stack recommended by the teaching assistant: bare-metal Ubuntu 26.04 with KVM/QEMU and libvirt/virt-manager, running the CAPEv2 automated malware-analysis sandbox. Bare-metal virtualization was chosen after nested virtualization (VirtualBox inside a VM, and VirtualBox under WSL2) proved unstable.

○ Components

- Host: Ubuntu 26.04, dual-booted alongside Windows on the internal drive.
- Hypervisor: KVM/QEMU driven through libvirt and virt-manager.
- Sandbox controller: CAPEv2 (installed at /opt/CAPEv2), with its cape-rooter, cape-processor, cape-web and cape services, backed by PostgreSQL and MongoDB.
- Victim VM: 'victim-win11' running Windows 10, hardened by disabling Windows Update and Windows Defender, with a local (non-Microsoft) account. The in-guest CAPE agent (agent.py) listens on port 8000.

- Network: an isolated libvirt virtual network (192.168.100.0/24, bridge virbr1). The victim VM has a static IP 192.168.100.150; the host is the gateway at 192.168.100.1. The network has no forwarding to the outside, guaranteeing containment.
- Internet simulation: INetSim on the host, bound to 192.168.100.1, provides fake HTTP/HTTPS/FTP/SMTP/etc. services so that samples receive responses instead of connecting to real infrastructure.
- **Analysis workflow**

For each sample, CAPEv2 reverts the victim VM to a clean snapshot, injects and launches the sample, monitors its API calls via in-guest hooking, captures all traffic crossing virbr1 with tcpdump, then reverts the VM and generates a JSON report. A single execution therefore yields both feature types (API sequence and PCAP) for the same run, as the assignment requires.

3. Dataset

Samples were obtained from MalwareBazaar (abuse.ch), where each sample is tagged with its malware family and from VirusShare. All samples are 32-bit or .NET Windows PE executables; non-PE types (scripts, documents) were excluded because the sandbox is configured for the PE analysis package. Each sample is tracked in labels.csv with columns filename, family, and MD5; the MD5 is the key used to match a sample to its CAPE analysis task.

After execution, five labelled samples across three families produced usable behavioural traces (at least 50 API calls). Several samples failed to execute — most because their PE manifest requires administrator elevation (Windows Error 740) while the analyzer runs as a standard user, and one RedLine sample bailed on repeated attempts. Failed runs are excluded from training.

Family	Sample	Task	API calls	DNS queries	Status
--------	--------	------	-----------	-------------	--------

AgentTesla	agenttesla_02.exe	10	15,603	40	Used (train)
AgentTesla	agenttesla_03.exe	17	1,797	39	Used (train)
Formbook	formbook_01.exe	11	8,524	12	Used (train)
Formbook	formbook_02.exe	12	8,824	11	Used (train)
RedLine	redline_01.exe	13	10,319	33	Used (train)
AgentTesla	agenttesla_01.exe	9	0	-	Failed (Error 740)
RedLine	redline_02 / 03	14/16/18	0	-	Failed (did not run)
unknown	sample01.exe	3/7/8/1/5	~4,000	6-10	Held out (test)

4. Feature Extraction

Feature extraction is implemented in `extract_features.py`. It reads `labels.csv`, locates each sample's CAPE analysis by matching the report's MD5, identifies the malware's own process (by matching the submitted filename, which isolates the sample's behaviour from unrelated operating-system processes), and produces the two feature groups below, writing one JSON file per sample plus a combined `dataset.json`.

- **Step 1 - API call sequence**

From the malware process's monitored calls, the extractor records the ordered sequence of Windows API names (e.g. `NtCreateFile`, `RegOpenKeyExW`, `InternetOpenA`). The ordered sequence is retained so that the classifier can exploit short-range ordering (which call follows which).

- **Step 2 - network features**

Two complementary network views are extracted from the same run. (a) A numeric traffic vector from the CAPE report: counts of UDP/TCP/ICMP/HTTP packets, number of DNS queries, and the number of unique destination ports and IPs. (b) The DNS query names, parsed directly from the run's dump.pcap with tshark. Domains belonging to known Windows/operating-system telemetry (e.g. *.microsoft.com, *.windows.com) are filtered out so that the network features reflect the sample's own behaviour rather than background OS noise.

Because DNS answers were not reliably served during collection, most samples' lookups went unanswered, so they did not proceed to full C2 connections; the recovered domain lists are therefore sparse. However, the volume of DNS activity is itself discriminative — AgentTesla issued ~40 DNS queries per run, RedLine ~33, and Formbook ~11–12 — and this numeric signal is retained as a network feature.

5. Hybrid Fusion Model

The classifier is implemented in classify.py using scikit-learn. It constructs two feature views and fuses them:

- API view: TF-IDF over API-call unigrams and bigrams, treating each API name as a token. Bigrams capture short call-ordering patterns; TF-IDF down-weights ubiquitous calls.
- Network view: the standardised numeric traffic vector concatenated with a TF-IDF encoding of the queried domain names.

The project uses early (feature-level) fusion: the API and network feature vectors are concatenated into a single combined feature space, on which a Random Forest classifier is trained. Early fusion lets the classifier learn cross-modal interactions directly. To quantify the value of fusion, three models are trained and compared under identical conditions: API-only, network-only, and the fused hybrid.

6. Experimental Setup and Results

Because the labelled dataset is small, evaluation uses Leave-One-Out cross-validation (LOOCV): each sample is tested once while the model trains on all remaining samples. All feature vectorizers and the scaler are fit inside each fold, on the training split only, to avoid information leakage. Table 2 reports LOOCV accuracy for the three models.

Feature View	LOOV accuracy
API sequence only	0.8
Network only	0.8
Fused (Hybrid)	0.8

The three views score identically at 0.80 because, on the current dataset, the single misclassification is structural rather than model-dependent: the RedLine class has only one usable sample, so under LOOCV the fold that holds out that sample leaves the model with no RedLine example to learn from, making that one error unavoidable for every feature view. In other words, the ceiling here is imposed by class imbalance (one singleton class), not by the fusion method. A balanced dataset with at least two samples per family is required before the fused model can demonstrate a measurable advantage over the single-view baselines; this is the primary avenue for improvement.

7. Discussion and Limitations

- Dataset size and balance. The usable set is 5 labelled samples across 3 families (AgentTesla $\times 2$, Formbook $\times 2$, RedLine $\times 1$). This is sufficient to demonstrate a correct, leakage-free pipeline but too small for statistically meaningful accuracy; the singleton RedLine class in particular caps LOOCV accuracy
- Execution failures. Several samples did not run — chiefly because their manifest requires administrator elevation (Error 740) while the analyzer runs as a standard user. Disabling UAC in the victim VM (and re-taking the clean snapshot) would let elevation-requiring samples execute and enlarge the dataset

- Sparse network features. With DNS unanswered during collection, samples largely did not reach their C2 endpoints, so domain-level features are thin and the network view leans on traffic-volume statistics. Serving synthetic DNS/HTTP responses (e.g. a dnsmasq catch-all, or a correctly functioning INetSim DNS) would produce richer, more discriminative network traffic
- Model choice. A Random Forest on TF-IDF features is a strong, interpretable baseline appropriate to the data scale. Sequence models (e.g. n-gram language models, LSTMs, or Transformers over the API stream) would better exploit long-range call ordering given more data

8. Conclusion

This project delivers a complete, working hybrid malware-classification pipeline: an isolated CAPEv2 sandbox that captures both API-call sequences and network traffic from a single execution, an extractor that turns those into fused feature vectors, and a classifier that compares API-only, network-only, and fused models under leakage-free cross-validation. The methodology satisfies all three required steps; the current numerical result (0.80) is bounded by dataset size rather than by the approach.